

Project Title: Hospital Management System

Project team members:

- Sheheb Ali Al-gumae – 2311504278
- Yılmaz Sönmez – 2311504267
- Heba K I Rusrus – 2211504207
- Muhammad Hamza Iqbal – 2311504308
- Beyhan Bahar Aslan – 2111504043

1 - Scenario

Overview: This system is designed to manage personal details of patients, appointment records of patients, status and availability of doctors, and treatments-bills associated with the appointment of patient in an hospital. Patients can have multiple emergency contact person. Allergies, blood types of patients also stored. Patients can make appointments with doctors from different departments. Departments are stored and each department has one head doctor, and multiple doctor working under it. Each appointment can have a unique pair of doctor and schedule. A doctor can have multiple schedule, more than one specialization, and only one job status. Treatments can be created from appointments. Treatments and bills cannot be without appointments. Each bill is associated with a payment method. Address of patients and doctors stored in detail.

Department: Each department has a unique Department ID, unique department name, unique telephone number, description, and a head doctor. Department ID is the key attribute. A department can have multiple doctors.

Doctor: Each doctor has a unique Doctor ID, first name, last name, birth date, office number, unique phone number, unique email, gender, marital status, job status id, department id and an address id. Doctor ID is the key attribute. A doctor can be head of one department.

Schedule: Each schedule has a unique Schedule ID, date and time. Schedule ID is the primary key. A schedule can be taken by multiple doctors. Active status stored in doctor-schedule relationship.

Job Status: Each job status has a unique Job Status ID and unique status name. Job Status ID is the primary key.

Specialization: Each specialization has a unique Specialization ID, and unique name. A doctor can have multiple specializations.

Patient: Each patient has a unique Patient ID, first name, last name, birth date, gender, unique phone number, unique email, blood id and an address id. Patient ID uniquely identifies each patient. A patient can book multiple appointments.

Blood Type: Each blood type has a unique Blood ID and unique type_name. Blood ID is unique identifier. Patients can have same blood type.

Allergy: Each allergy has a unique Allergy ID and unique ICD10 code. Allergy ID alone is the primary key. Patients can have same allergy.

Emergency Person: Each emergency person has a unique Emergency ID, first name, last name and a unique phone number. Emergency ID is the unique identifier. A emergency person can be associated with multiple patients.

Address: Each address has a unique Address ID, country, province, district, neighbourhood and postal code. Address ID is the unique identifier. A address is for one patient or one doctor.

Appointment: Each appointment has a unique Appointment ID, status, patient, doctor and schedule. Appointment ID uniquely identifies each appointment. A appointment is for one patient.

Treatment: Each treatment has a unique Appointment ID and an ICD10 code. Appointment ID and ICD10 code together forms the identifier. Each treatment belongs to one appointment and treatment can't exists without appointment.

Bill: Each bill has a unique Bill ID, ICD10 code total amount, tax, payment id, and is paid. Bill ID and ICD10 code together forms the primary key. Each bill belongs to one treatment.

Payment Method: Each payment method has a unique Payment ID and unique type_name. Payment ID is the unique identifier.

Relationships: A department can have more than one doctor; a doctor can work in only one department (1:N). A department can have one head doctor; one doctor can be head of one department (1:1). A doctor can have one job status; a job status can be for multiple doctors (1:N). A doctor can have multiple specializations; each specialization can be taken by multiple doctors (M:N). A doctor can have one address; a address can have one doctor (1:1). A doctor can take multiple schedules; each schedule can have multiple doctors (M:N). A doctor can be assigned to multiple appointment; a appointment can be taken by one doctor (1:N). A patient can have one address; a address can have one patient (1:1). A patient can have one blood type; a blood type can be in multiple patients (1:N). A patient can have multiple allergies; a allergy can be in multiple patients (M:N). A patient can have multiple emergency person; each emergency person can have multiple patients (M:N). A appointment can be booked by one patient; a patient can book multiple appointments (1:N). A appointment can include one schedule; a schedule can be for multiple appointments (1:N). A treatment can be created from one appointment; a appointment can create multiple treatments (1:N). A bill can be created from one treatment; a treatment can create one bill (1:1). A bill can have one payment method; each payment method can have multiple bills (1:N).

Scenario Corrections

- Fixed mistakenly written (M:N) as (1:N) in the relationship Treatment-Appointment.
- "payment method" in Bill table corrected as "payment id".
- "blood type" in Patient corrected as "blood id".
- "job status" in Doctor corrected as "job status id".
- "address" in both Doctor and Patient corrected as "address id".
- Gender table removed and instead 1 letter gender attribute placed into Doctor and Patient. It was unnecessary.
- Added new text line to the Schedule table description.

2 - Normalization

Address Table (1NF)

<u>address_id</u>	country	province	district	neighbourhood	postal_code
1	Turkiye	Balikesir	Bandirma	Kayacik	10200
2	England	Manchester	Stockport	Cheadle	SK8 2NP
3	Turkiye	Balikesir	Bandirma	Levent	10200
4	Turkiye	Istanbul	Uskudar	Icadiye	34664

Problems

- Have to insert same columns every time.
- If you update country, you would end up updating all other columns.
- If user with address_id=2 deleted, we will lost England country data.

Functional Dependencies

- address_id → country (partial dependency)
- address_id → province (partial dependency)
- address-id → neighbourhood (partial dependency)
- neighbourhood → district (transitive dependency)
- district → province (transitive dependency)
- province → country (transitive dependency)
- address_id, district → postal_code (key)

Second Normal Form (2NF)

<u>address_id</u>	country	province	neighbourhood
1	Turkiye	Balikesir	Kayacik
2	England	Manchester	Cheadle
3	Turkiye	Balikesir	Levent
4	Turkiye	Istanbul	Icadiye

<u>address_id</u>	<u>district</u>	postal_code
1	Bandirma	10200
2	Stockport	SK8 2NP
3	Bandirma	10200
4	Uskudar	34674

Problems

- We cannot insert values without address_id.
- If we delete address_id, we will delete all data with it.

Third Normal Form (3NF)

Create new tables from transitive dependencies and separate table for country. Add them their own id.
(Table names written below each table.)

<u>country_id</u>	country_name
1	Turkiye
2	England

Country

<u>province_id</u>	province_name	country_id
1	Balikesir	1
2	Manchester	2
3	Istanbul	1

Province

<u>district_id</u>	district_name	province_id
1	Bandirma	1
2	Stockport	2
3	Uskudar	3

District

<u>address_id</u>	neighbourhood_id
1	1
2	2
3	3
4	4

Address

<u>neighbourhood_id</u>	neighbourhood_name	district_id	postal_code
1	Kayacik	1	10200
2	Cheadle	2	SK8 2NP
3	Levent	1	10200
4	Icadiye	3	34674

Neighbourhood

New relationships after normalization: A address can have one neighbourhood; a neighbourhood can be for multiple address (1:N). A neighbourhood can be in one district; a district can have multiple neighbourhoods (1:N). A district can be in one province; a province can have multiple districts (1:N). A province can be in one country; one country can have multiple provinces (1:N).

Treatment Table (3NF)

<u>appointment_id</u>	<u>icd10_code</u>
1	XHF
2	XHF
1	DDA

The table is already in 3NF form, but still there are things we need to consider.

Problems

- If appointment_id is gone, we will lose icd10 code as well.
- You don't know if icd10 code is correct or not. Wrong data insertion problem.
- You may have to write same icd10 code for different appointments.

Functional Dependencies

There is none. You cannot get [appointment → icd10_code] or [icd10_code → appointment_id].

Solution

Create separate table for icd10 codes and give it its own id. Use treatment_id as identifier for Treatment table and take icd10_id and appointment_id as foreign keys. This way Treatment table becomes a form of connection table between Appointment and ICD10 table.

<u>treatment_id</u>	appointment_id	icd10_id
1	1	1
2	2	1
3	1	2

Treatment

<u>icd10_id</u>	icd10_code	description
1	XHF	text1
2	DDA	text2

ICD10

New relationship after normalization: A treatment can have one icd code; a icd code can be for multiple treatments (1:N).

Bill Table (1NF)

<u>bill_id</u>	<u>icd10_code</u>	total_amount	tax	payment_id	is_paid
1	XHF	100	20	null	false
2	XHF	110	30	2	true
3	DDA	200	50	1	true

At first, we have to change icd10_code as treatment_id. Because we have created a separate table for it, and a Bill must be for a Treatment.

Second Normal Form (2NF)

<u>bill_id</u>	<u>treatment_id</u>	total_amount	tax	payment_id	is_paid
1	1	100	20	null	false
2	2	110	30	2	true
3	3	200	50	1	true

Problems

- bill_id and treatment_id are always same. Duplicate column problem.

Solution

Simply delete bill_id and make treatment_id primary key. This way Bill table will become weak and can't be without Treatment table.

Third Normal Form (3NF)

Bill

<u>treatment_id</u>	total_amount	tax	payment_id	is_paid
1	100	20	null	false
2	110	30	2	true
3	200	50	1	true

Relationship remains same.

Allergies (3NF)

<u>allergy_id</u>	icd10_code
1	ABC
2	ABD

Here table is already in 3NF, but we need to change icd10_code as icd10_id.

Final table:

Allergies

<u>allergy_id</u>	icd10_id
1	10
2	11

New relationship after normalization: A allergy can have one icd code; a icd code can define one allergy (1:1).

3 - Entity Relationship (ER) Diagram

After performing normalization steps, ER diagram is created.

